

Q3 Azure Table Schema Migration Playbook

Jake feedback edition · smallest-change guide · 28 September 2026

CURRENT STATUS Runtime migration verified · Profile naming fix committed as 894f37d8 · API 139/139

1. Jake 피드백을 한 문장으로 해석하기

기존 PostgreSQL schema의 이름과 의미는 가능한 한 유지한다. Azure Table의 구조적 한계 때문에 꼭 필요한 부분만 바꾸고, 외부 API 동작은 그대로 유지하며, 달라진 부분과 이유를 Jake가 검토할 수 있도록 명확히 기록한다.

CHECK 목표는 PostgreSQL 물리 구조 복제가 아니라 데이터 의미와 API 계약 보존이다.

2. 변경 여부를 결정하는 우선순위

1. 이름을 그대로 쓸 수 있으면 그대로 쓴다: internal_id, external_id 같은 기존 property 이름을 보존한다.
2. Azure가 직접 지원하지 않는 경우에만 바꾼다: PartitionKey/RowKey, 보조 index entity, JSON 문자열 저장.
3. 변경 후에도 handler 응답과 오류 의미가 기존 API와 같은지 테스트한다.
4. 의도적 차이와 아직 해결하지 않은 gap을 문서에 따로 적는다.
5. 한 번에 한 repository만 바꾸고 build → repository test → 139 API test 순서로 확인한다.

3. 실제 PostgreSQL → Azure Table 매핑

profile → Profiles + ProfileByExternalId

Profiles는 internal_id로 canonical profile을 찾고, ProfileByExternalId는 external_id lookup을 제공한다. PostgreSQL의 secondary lookup을 Azure Table에서 구현하기 위해 두 entity가 필요하다.

question → QuestionData / RowKey=question

한 질문을 하나의 partition으로 두어 질문 변경과 audit event를 같은 Azure transaction에 넣는다.

questionAnswer → QuestionData / RowKey=answer:{profileId}:{createdAt}:{id}

답변을 질문 partition에 같이 두어 질문별 조회 범위를 제한하고 profile별 최신 답변 규칙을 유지한다.

logQuestion + questionAction → QuestionData / RowKey=event:{id}

eventKind로 log와 action을 구분한다. PostgreSQL의 두 테이블은 합쳐지지만 API에서 필요한 의미는 보존한다.

questionShare → QuestionData / RowKey=share:{id}

공유 entity는 질문 partition에 저장한다. 현재 PostgreSQL의 type 컬럼은 Azure entity에 없으므로 사용 여부를

확인해야 한다.

FollowUp/QuestionShare command·event·filter → WorkflowData

correlationId 또는 filter id를 partition 기준으로 사용해 command 완료와 event 생성을 가능한 범위에서 함께 처리한다.

4. 그대로 유지하는 것

- 새 Profile entity는 internal_id와 external_id를 그대로 저장한다.
- 파일명, exported function 이름, handler 호출 방식, API 응답의 internal_id/external_id는 바꾸지 않는다.
- 기존 UUID 값과 createdAt의 의미를 유지한다.
- 질문·답변·공유·command의 기존 id, status, action, correlationId 의미를 유지한다.
- 기존 camelCase Azure profile entity는 read fallback으로 계속 읽는다.
- ProfileByExternalId만 남은 partial write는 다음 ensureProfile 호출에서 Profiles 행을 복구한다.

5. Azure 제약 때문에 의도적으로 달라지는 것

- 11개 관계형 테이블을 4개 Azure Table로 합친다. Azure에서는 query access pattern과 partition이 우선이다.
- Foreign Key가 없으므로 repository가 profile/question 존재 여부를 확인한다.
- JSON, 배열, JSON Patch는 Azure 지원 property 타입에 맞춰 JSON 문자열로 저장하고 경계에서 복원한다.
- PostgreSQL timestampz는 ISO-8601 문자열로 저장한다. API 소비자가 같은 의미로 읽을 수 있어야 한다.
- 트랜잭션은 같은 table·같은 PartitionKey에 한정되고 최대 100 operations / 4 MiB이다.
- ETag를 사용해 동시에 같은 질문을 수정할 때 조용히 덮어쓰는 것을 막는다.

6. 의도적 차이가 아닌 실제 gap

CAUTION 아래 항목은 'Azure라서 어쩔 수 없음'으로 숨기지 말고 Jake에게 상태와 결정을 알려야 한다.

- questionShare.type: PostgreSQL에는 있지만 현재 Azure share entity에는 없다. API에서 사용되는지 확인 후 보존 또는 명시적 제위가 필요하다.
- 기존 데이터 backfill: 로컬 PostgreSQL의 기존 profile/question/answer/log는 현재 Azurite에 같은 ID로 존재하지 않는다.
- Profile 두 테이블: cross-table atomic transaction은 불가능하다. 현재는 다음 호출의 self-repair로 실패 창을 줄인다.
- 기존 Azure camelCase entity: read fallback은 있지만 자동으로 snake_case로 rewrite하지 않는다.
- QuestionData 목록 조회: profile secondary index가 없어 현재 전체 table scan이다.
- 100명 초과 공유: 여러 transaction으로 나뉘므로 전체 작업이 원자적이지 않다.

7. Smallest-change 실행 순서

1. 변경하려는 PostgreSQL table의 실제 column·nullability·default·constraint를 read-only로 확인한다.
2. 현재 handler가 실제로 읽고 쓰는 필드만 추적한다. 사용하지 않는 필드를 임의로 삭제하지 않는다.

3. Azure entity property는 기존 이름을 우선 사용한다. 이름 변경이 필요하면 이유를 한 줄로 기록한다.
4. repository 한 곳만 변경하고 그 repository의 작은 계약 테스트를 추가한다.
5. 전체 API 139개와 실행 전후 Azure entity count를 확인한다.
6. Sequelize 파일 삭제는 기능 migration을 방해하지 않도록 별도 cleanup commit으로 처리한다.
7. Terraform apply 전에는 반드시 plan에서 기존 PostgreSQL resource 삭제 여부를 검토한다.

8. 현재 검증 명령

```
# repository root

pnpm exec vitest --root . run \
  module/quest3Tier/func/vitest/profile.test.mjs \
  module/quest3Tier/func/vitest/tableQuestion.test.mjs \
  --pool=forks --no-file-parallelism

# Function Host가 실행 중일 때

pnpm exec vitest --root . run \
  module/quest3Tier/func/vitest/testQuestion.test.mjs \
  --pool=forks --no-file-parallelism
```

현재 기준 결과: Profile 5/5, Azure repository 18/18, HTTP API 139/139. 테스트 전후 Profiles, ProfileByExternalId, QuestionData, WorkflowData entity count가 실제로 증가해야 한다.

9. Jake에게 차이를 설명하는 형식

Keep

We kept the existing field names and API contract wherever Azure Table supports them directly.

Change

We changed the physical layout only where Azure Table requires a different partition/index strategy.

Why

Azure Table has no joins, foreign keys, arbitrary secondary indexes, or cross-table transactions.

Proof

The existing API tests pass, repository tests cover the new storage behaviour, and table entities change during the run.

Known gaps

List every deferred field, non-atomic boundary, scan, and data-backfill decision explicitly.

10. Definition of Done

- Q3 handlers and services use only the TypeScript Azure Table repositories at runtime.
- Existing field names are preserved unless an Azure limitation requires a documented change.
- Existing API response and error behaviour remain compatible for tested flows.
- Repository tests and 139 HTTP API tests pass.
- Azure Table entities change during the test run.
- No generated sibling .mjs/.d.mts remains beside an .mts source.
- Known schema differences and deferred risks are included in the PR description.

CAUTION Not automatically included: production data backfill, physical PostgreSQL schema parity, or destructive Terraform apply. Each requires explicit confirmation.